

Dokumentasi Proyek: Generator Landing Page Cerdas

1. Konsep & Tujuan Proyek 🎯

Tujuan utama proyek ini adalah untuk menciptakan sebuah sistem "**Pabrik Konten**" yang berjalan secara terpisah dari website utama. Sistem ini secara otomatis menghasilkan potongan-potongan konten halaman arahan (landing page) yang modular, kaya akan gaya, dan diisi oleh Kecerdasan Buatan (AI).

Arsitektur terpisah ini **sangat disarankan** karena memisahkan proses kreatif dan generasi konten yang intensif dari website WordPress yang sudah berjalan, menjaga stabilitas dan keamanan.

- **Pabrik Konten (Lokal):** Lingkungan Python di komputer Anda yang bertugas membuat file HTML.
- **Toko (Live):** Website WordPress Anda yang bertugas menampilkan HTML yang sudah jadi.

2. Tumpukan Teknologi 🛠️

Berikut adalah teknologi yang kita gunakan dalam proyek ini:

- **Bahasa Pemrograman: Python.** Dipilih karena ekosistemnya yang matang untuk AI, manipulasi data, dan scripting.
- **Mesin Template: Jinja2.** Digunakan untuk membuat "cetakan" HTML modular dan mengisinya dengan data secara dinamis.
- **Konten AI:** (model gemini-1.5-flash-latest). Bertugas sebagai "copywriter" yang menghasilkan konten teks dalam format JSON dan csv berdasarkan topik yang diberikan. AI menggunakan semua model free tier.
- **Desain & Styling: Template Sandbox (HTML + Tailwind CSS).** Digunakan sebagai sumber aset desain, komponen modular, dan styling.
- **Data Statis: File CSV.** Digunakan untuk mengelola data yang tidak perlu dibuat oleh AI, seperti link URL atau path gambar yang sudah pasti.
- **Platform Target: WordPress.** Sistem Manajemen Konten (CMS) tempat hasil akhir HTML akan dipublikasikan. Kita menggunakan **Child Theme (Blocksy)** agar kustomisasi aman dari pembaruan.

3. Struktur Kode Python Modular

Untuk membuat proyek lebih terorganisir dan mudah dikembangkan (scalable), kita akan memecah logika dari satu file run.py menjadi beberapa modul dengan tanggung jawab masing-masing.

- **config.py:** Menyimpan semua konfigurasi, seperti API Key Gemini dan path file.

- **data_manager.py:** Berisi fungsi untuk membaca dan memproses file CSV (content_from_json.py bisa digabung di sini).
- **ai_generator.py:** Bertanggung jawab untuk membuat prompt dan berkomunikasi dengan Gemini API.
- **template_renderer.py:** Berisi fungsi untuk merender template Jinja2 dan menyimpan file HTML (content_from_html.py dan ai_json_html.py bisa digabung di sini).
- **main.py:** Skrip utama yang akan Anda jalankan. Ia akan mengimpor dan mengorkestrasi fungsi dari modul-modul lain.

4. Alur Kerja & Langkah-Langkah

Proses dari ide hingga menjadi halaman di WordPress mengikuti alur kerja berikut:

Langkah 1: Persiapan Aset & Template (Satu Kali)

1. **Template Modular:** Membedah file HTML dari template Sandbox menjadi komponen-komponen Jinja2 (hero.jinja, services.jinja, why_chose_us.jinja, price.jinja, faq.jinja, dll.) dan menyimpannya di folder templates/components/.
2. **Proses Build CSS:** Menjalankan npm run build pada folder asli Sandbox untuk menghasilkan style.css yang teroptimasi.
3. **Setup WordPress:**
 - Membuat dan mengaktifkan Child Theme.
 - Menyalin folder assets dan style.css yang sudah dioptimalkan ke dalam folder Child Theme.
 - Mendaftarkan semua aset (CSS & JS) melalui functions.php menggunakan wp_enqueue_style dan wp_enqueue_script.

Langkah 2: Eksekusi Generator (Per Halaman)

Ini adalah proses yang dijalankan oleh main.py dan mencakup diagram alur Anda.

1. **AI = JSON + CSV:**
 - main.py memanggil fungsi dari ai_generator.py dengan sebuah topik.
 - ai_generator.py membuat prompt dan meminta data konten ke Gemini API.
 - AI mengembalikan data dinamis dalam format JSON.
 - main.py memanggil fungsi dari data_manager.py untuk memuat data statis (link, gambar) dari file CSV.
2. **JSON dan/atau CSV = HTML:**
 - main.py menggabungkan data dinamis dari AI dengan data statis dari CSV.
 - Data gabungan ini kemudian dikirim ke template_renderer.py.
 - template_renderer.py menggunakan Jinja2 untuk merender template dengan data tersebut, menghasilkan satu "potongan" HTML yang siap pakai.

- Skrip juga menyimpan file .html dan .json dengan nama unik untuk arsip.

Langkah 3: Publikasi ke WordPress

1. **HTML = SEO Optimized:** Konten yang dihasilkan AI (judul, deskripsi) sudah diinstruksikan untuk SEO-friendly. Struktur HTML dari template juga bersih dan semantik.
2. **HTML SEO Optimized = WordPress:**
 - Buat Laman (Page) baru di WordPress.
 - Gunakan template halaman "Default" atau "Full Canvas".
 - Salin isi dari file .html yang dihasilkan dan tempel ke dalam blok "Custom HTML".
 - Gunakan plugin SEO (seperti Rank Math/Yoast) untuk menyempurnakan meta-data.
 - Terbitkan.

5. WORDPRESS & ENRICHMENT

Proyek ini memiliki potensi besar untuk dikembangkan lebih lanjut.

- **Automasi Penuh dengan WordPress REST API (aiwordpress.py):** Ini adalah langkah pengembangan paling logis. Modul aiwordpress.py bisa dikembangkan untuk berinteraksi dengan WordPress REST API. Setelah HTML dibuat, skrip bisa langsung **membuat dan mempublikasikan halaman di WordPress secara otomatis**, menghilangkan langkah copy-paste.
- **Enrichment Konten (aicontent_enrichment.py):** Modul ini bisa dikembangkan untuk melakukan panggilan AI kedua. Setelah konten dasar dibuat, modul ini bisa "memperkaya" teks tersebut, misalnya dengan menambahkan data statistik, kutipan relevan, atau membuat variasi tulisan untuk A/B testing.
- **Manajemen Topik Massal:** Kembangkan main.py dan data_manager.py untuk membaca daftar topik dari sebuah file CSV, lalu melakukan perulangan untuk menghasilkan puluhan atau ratusan halaman dalam sekali eksekusi.
- **A/B Testing Varian Komponen:** Perluas logika pemilihan acak komponen (random.choice()) untuk menciptakan berbagai kombinasi layout halaman secara otomatis untuk tujuan A/B testing.
- **Integrasi Gambar Dinamis:** Kembangkan modul untuk berinteraksi dengan API stok foto (Unsplash) atau AI generator gambar untuk mencari dan menyisipkan gambar yang relevan dengan topik secara otomatis.